Version 2026-02-11

2110524 Cloud Computing Technologies Midterm Project “Infrastructure-as-Code”

You are going to automatically deploy a Wordpress website on Amazon web services using an infrastructure-as-code software called OpenTofu.

1. You should have a basic idea on how to setup wordpress from the VPC activity
2. Download and setup OpenTofu on your computer or EC2 instance (https://www.opentofu.org/)
3. In this project, you will create a OpenTofu script that will deploy and provision
    a. A new VPC for your project. You may **not** use the default VPC in the region.
    b. An EC2 free tier instance, with proper configurations, for installing and running the Wordpress app (https://wordpress.org/download/).
    c. An RDS free tier instance running MariaDB.
    d. A new S3 bucket, with proper settings, and configure Wordpress to use it for media storage via WP Offload Media plugin (https://aws.amazon.com/th/blogs/compute/deploying-a-highly-available-wordpress-site-on-amazon-lightsail-part-2-using-amazon-s3-with-wordpress-to-securely-deliver-media-files/).
    e. Various other services/configurations (IAM User, VPC, Internet Gateway, security groups, etc.) necessary to make this a fully functioning Wordpress web app on AWS.

Read these documents on how to set up Wordpress with S3.

1. https://aws.amazon.com/th/blogs/compute/deploying-a-highly-available-wordpress-site-on-amazon-lightsail-part-1-implementing-a-highly-available-lightsail-database-with-wordpress/
2. https://aws.amazon.com/th/blogs/compute/deploying-a-highly-available-wordpress-site-on-amazon-lightsail-part-2-using-amazon-s3-with-wordpress-to-securely-deliver-media-files/
3. https://deliciousbrains.com/wp-offload-media/doc/settings-constants/
4. https://deliciousbrains.com/wp-offload-media/doc/iam-roles/

Note : The goal of the project is to be able to automate deployment from scratch (source code), i.e., continuous deployment (CD). **You are not allowed to deploy Wordpress using any other means**. So, no docker image, Wordpress appliance, or snap package manager are allowed. Doing so would result in getting a score of 0 for this project. However, feel free to study those provisioning scripts for ideas. Additionally, you may want to set up Wordpress on an EC2 instance manually to see what you need to configure after installation.

Additional requirements:

1. Your script shall accept the `terraform.tfvars` file and configure the region, availability_zone, bucket_name, etc. accordingly (See : https://gist.github.com/theminer3746/150c52809e0235e99265ee1e252912cf).
   If you need to configure your own variables, separate them into another file called `private.tfvars` and use multiple `-var-file=` arguments in `tofu apply`.
2. For this project, we will use the **Ubuntu Server 24.04 LTS (HVM)** AMI. Because different regions use different AMI ID's for the same image, a single AMI ID can not be used across regions. You will need to specify the AMI ID in your configuration file (see above).
3. You must use PHP version 8.3 and MariaDB version 11.8.5. The version number must be explicitly specified within your installation script.
   Do NOT use 3rd party repositories for PHP such as ppa:ondrej/php.
4. **Do NOT** use the remote-exec provisioner.
5. You MUST use the IAM role for S3 authentication. Do **NOT** use aws_iam_access_key.
6. You should take security into consideration when creating security groups. Think of all the scenarios thoroughly. For example, the database instance should not be accessible by anyone on the Internet, and should only be accessible by the app instance's IP/subnet only. This translates to only allow incoming TCP connections to port 3306 (MariaDB's port) on the database instance from the app instance's IP/subnet only.
7. You **MUST** name your subnet using the `Name` property in `tags` according to the subnet names in the diagram below.
8. Note that your AWS keys should never be included in the OpenTofu script. Anyone who has set up the proper `~/.aws/credentials` or roles should be able to run your script. **Do NOT** hardcode access key/IAM role ID.
9. Make sure to add this ssh key to your `~/.ssh/authorized_keys` to the Wordpress instance so I can connect to your wordpress installation `ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIODaHqtrCOBpfD+meWggDG5gFEqnNDtpxnqQ7xWIfXfL cloud-wordpress`
10. Your script should display the elastic IP address after running `tofu apply`.

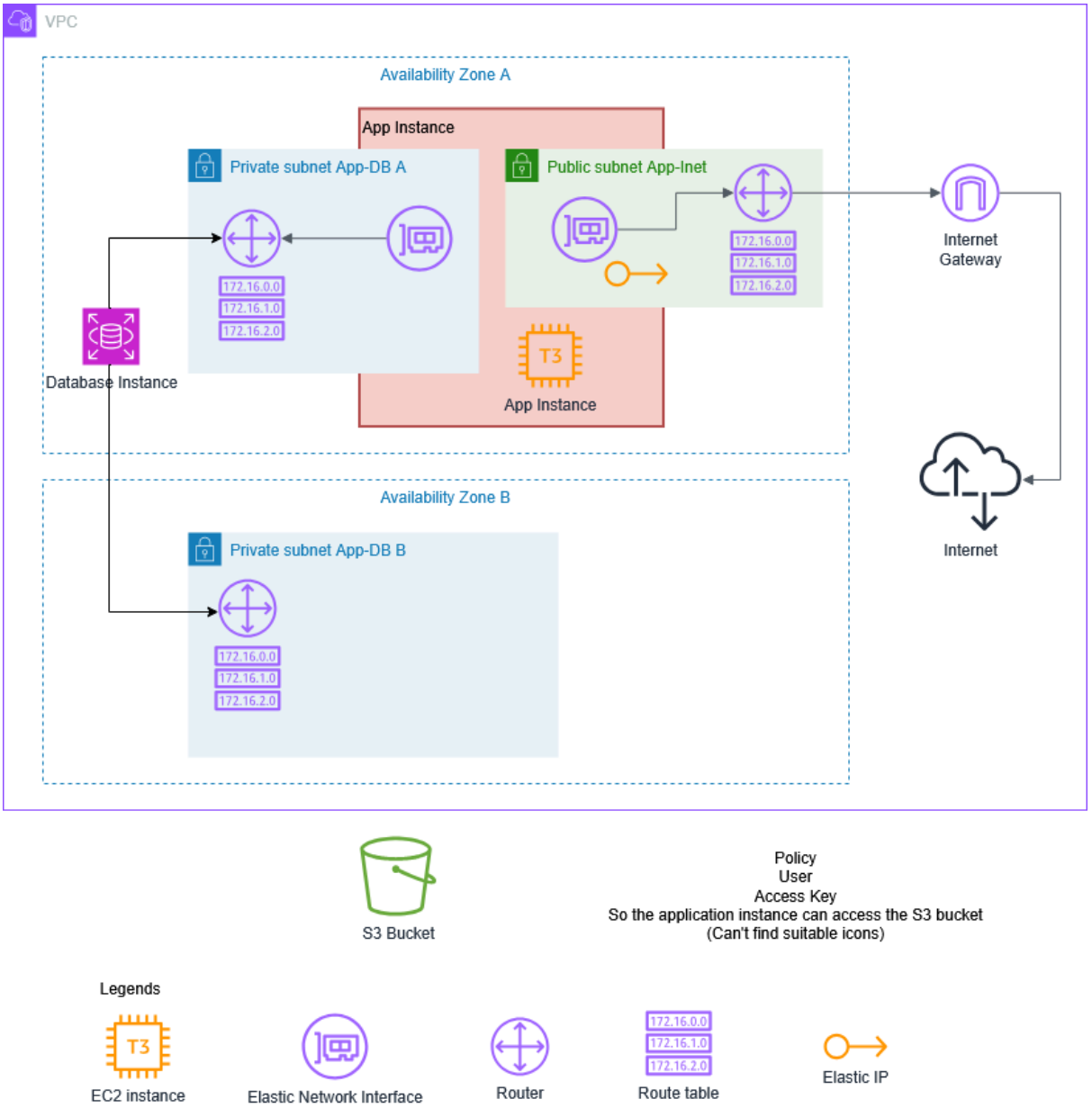


Figure 1: Your infrastructure should look like this after running tofu apply. Any unfamiliar services will require additional self-study.

Diagram explanation:

1. An EC2 instance (App) running Wordpress with 2 elastic network interfaces.
    a. Interface 1 has an elastic IP attached and is in a publicly accessible subnet associated with a routing table and an Internet gateway.
    Users will connect to Wordpress using the elastic IP on this interface.
    b. Interface 2 is in a private subnet used only to communicate with the database instance.
2. An RDS instance running MariaDB.
    a. AWS requires the RDS instance to be connected to 2 subnets even though we only need one. Hence, this is why Private subnet App-DB B exists.

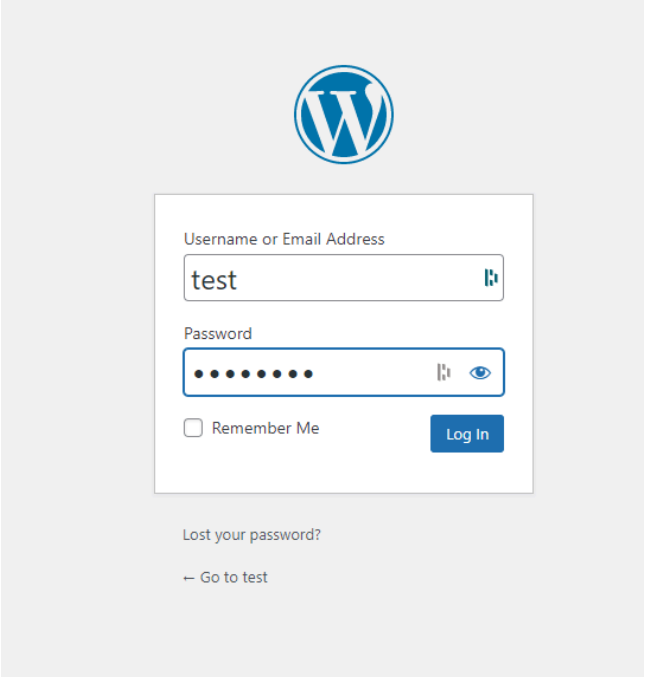
OpenTofu resources that you <u>may</u> need to use (in no particular order) :

1. aws_vpc
2. aws_subnet
3. aws_network_interface
4. aws_security_group
5. aws_security_group_rule
6. aws_instance
   Note : You may use the user_data property to pass the application provisioning script to the instance (see https://search.opentofu.org/provider/hashicorp/aws/latest/docs/resources/instance#argument-reference)
7. aws_db_instance
8. aws_db_subnet_group
9. aws_internet_gateway
10. aws_route
11. aws_route_table
12. aws_route_table_association
13. aws_eip
14. aws_s3_bucket
15. aws_iam_user
16. aws_iam_policy

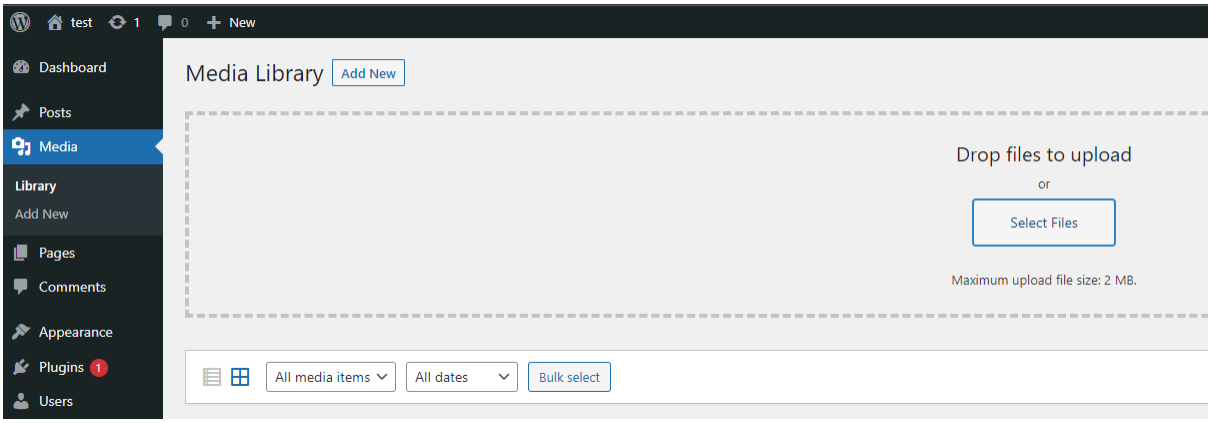
This is not a definitive list. You can use resources not in this list if you want.

Expected results

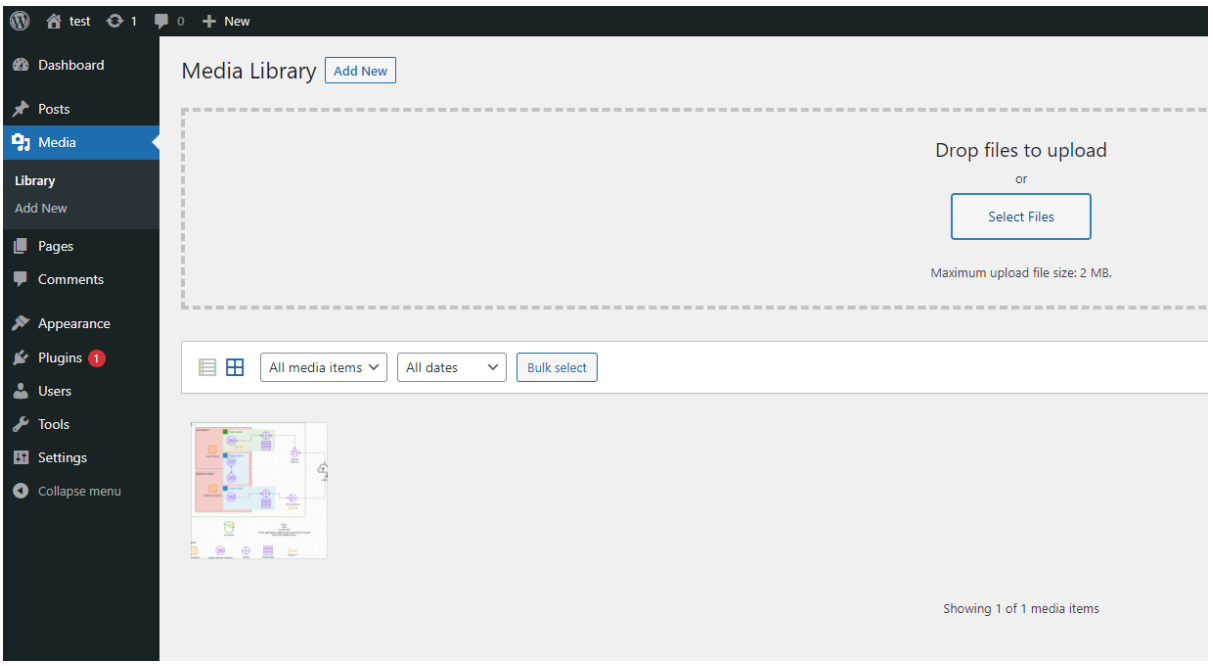
After running `tofu apply`, you should be able to login to the admin panel via http://<your-elastic-ip>/wp-admin



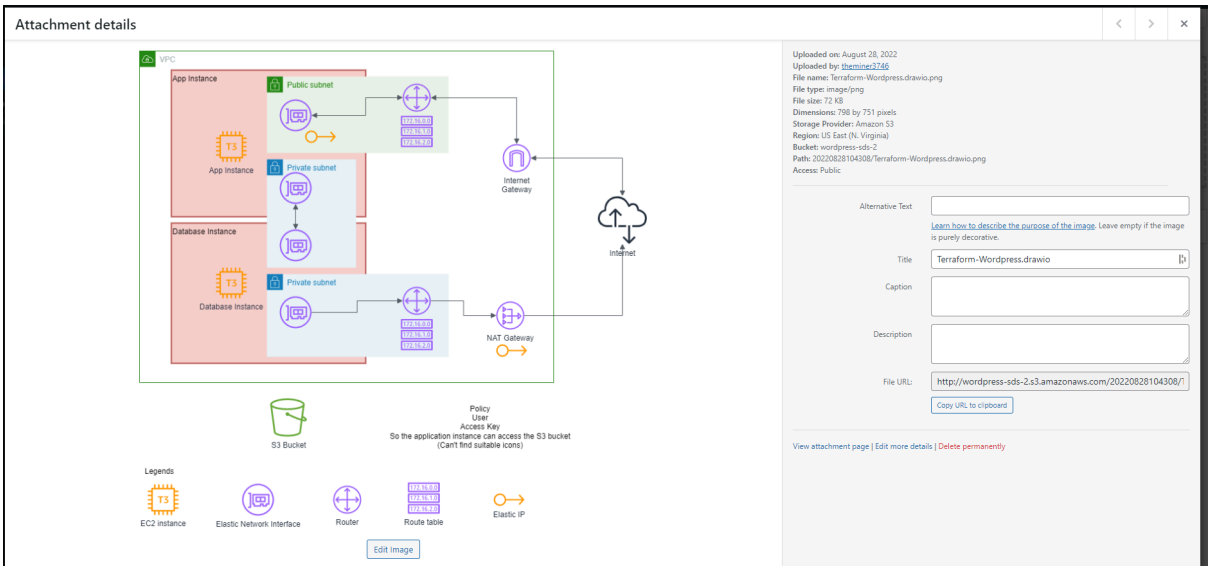
Version 2026-02-11



You should be able to upload media files to wordpress.



Click the uploaded media.



Every file should have their url changed to s3 url and be publicly accessible such as http://wordpress-sds-2.s3.amazonaws.com/20220828104308/Terraform-Wordpress.drawio.png

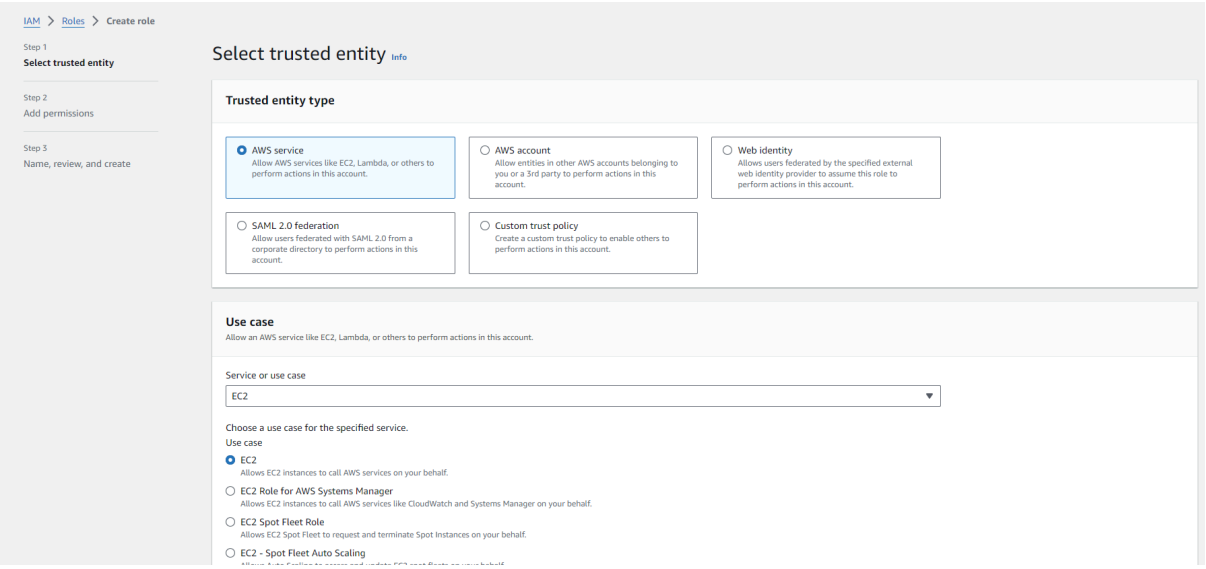
Additional hints that may be useful

1. Take a look at cloudinit. It is a powerful provisioning tool. You can also use a plain bash script if you prefer.
2. If you use cloudinit, look at `/var/log/cloud-init.log` and `/var/log/cloud-init-output.log` for execution results/errors.
3. Many tutorials use Apache as the web server. However, if you prefer Nginx, feel free to go ahead.
4. You can use `wp core install --url="${aws_eip.wordpress_ip.public_ip}" --admin_user="${var.admin_user}" --admin_password="${var.admin_pass}" --admin_email="exmaple@example.com" --title="Cloud" --skip-email` to configure the admin account from installation.

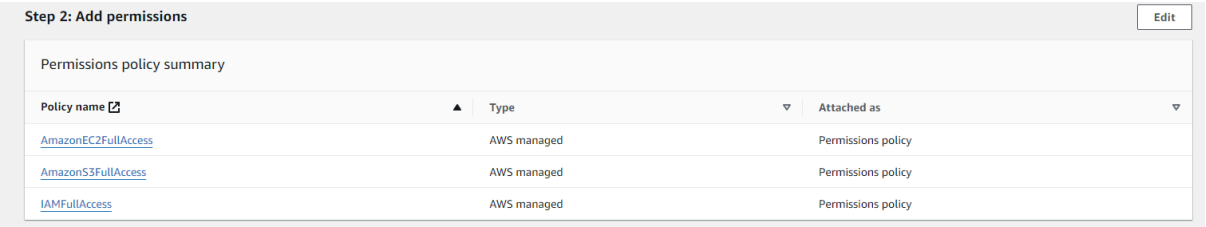
Setting up an EC2 instance to run OpenTofu with suitable policy

We recommend running OpenTofu on an EC2 instance instead of your own notebook to avoid ChulaWifi connectivity issues when demoing.

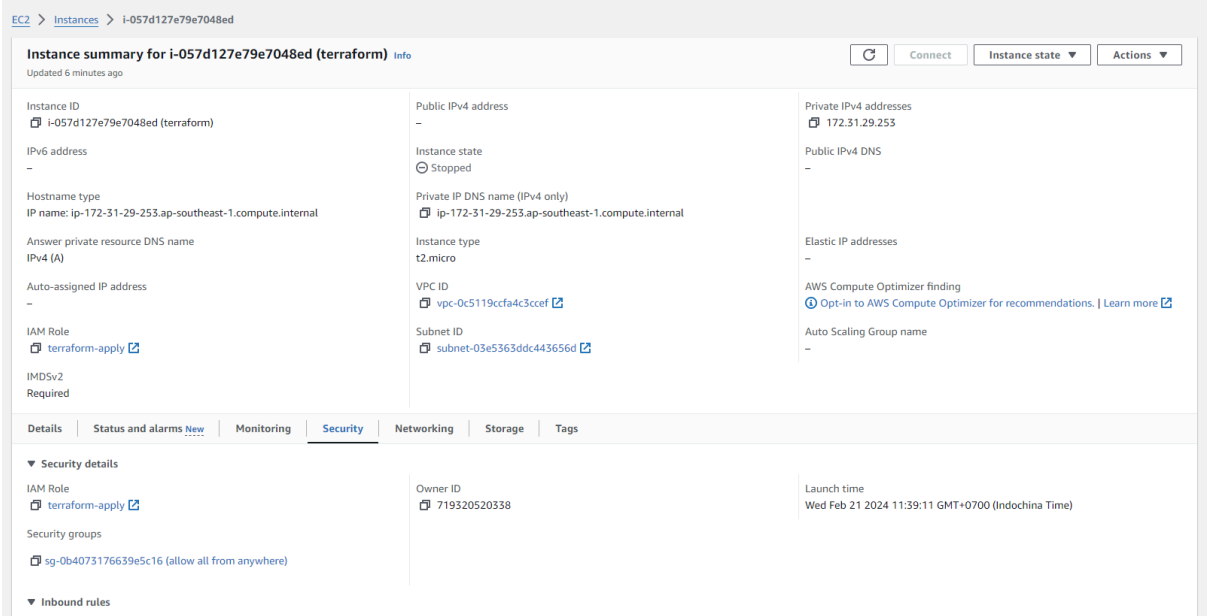
1. Create an instance for running OpenTofu provisioning. (This is not to be used for/by the web application directly.)
2. Follow OpenTofu's documentation on how to install it on your instance (see https://opentofu.org/docs/intro/install/)
3. Create an IAM role with AWS service trusted entity type and select EC2 as the use case.



4. In the Add permissions tab, select `AmazonEC2FullAccess`, `AmazonS3FullAccess`, and `IAMFullAccess`.



5. On the OpenTofu provisioning instance, attach the IAM role you created.



6. You can now use this instance to run `tofu`.

What to submit by the deadline

1. Your Tofu script. Do **not** include the `.terraform/` folder, `.terraform.lock.hcl`, `terraform.tfstate`, `terraform.tfstate.backup`.
2. A link to your youtube video (< 5 mins long) that
   a. Demo's your `tofu apply`
   b. Shows the instances and S3 bucket it provisions
   c. Demo's successful install of Wordpress and upload a file through the web UI
   d. Shows the objects in your S3 bucket after file upload

What to do on demo day (in class):

Instructors will assign you a region and you will change your `terraform.tfvars` file to configure your script to deploy resources in that region in your AWS account. You will run your Tofu script with the new variables. After the script is done, you will demo the same content as your recorded video, and answer some questions about what you have done. Note that you should test deploying your script on other regions to confirm you understand the process and changes required prior to demo day.

Version 2026-02-11

The demo grading checklist

	ชื่อ account aws ตรง
	ไม่มี instance ใน region นั้น
	ไม่มี bucket ชื่อนั้น
	แก้ไขชื่อ bucket ใน tfvars
	สั่ง tofu graph -type=plan
	สั่ง tofu apply
Pre run	เช็ค upload screenshot + graph ขึ้น mcv
	เปิดเว็บ wordpress ได้
	hostname ใน region ถูกต้อง (ให้นิสิตสั่ง nslookup ip wp)
	login หน้า admin
	ห้ามแก้ไข config ด้วยมือใน ui
	Media > Add New > Select Files อัพโหลดสำเร็จ
	คลิ๊กรูปที่ upload ขึ้นไป ดู attachment details File URL ต้องมาจาก S3
ตรวจใน web wordpress	File URL ต้องมาจาก S3 ตรงกับชื่อ bucket
ตรวจใน instance wordpress	Version ของ php ต้องเป็น 8.3 (สั่งรัน php -v)
	wordpress instance ip ตรงกับใน url
	Show CIDR public subnet App-Inet
	Show CIDR private subnet App-DB
	db instance ไม่มี public ip
	Show CIDR private subnet DB-Inet
	upload file แล้วเข้า S3 (ดูใน web console S3)
	vpc มี 3 subnet
	Show EC2 security group
ตรวจใน ec2 console	Show RDS security group